

StoreRate

Project Documentation & Coding Challenge Submission Guide

Purpose: Full-stack store rating platform developed as a software engineering coding challenge for Roxiler Systems.

Repository: <https://github.com/Ak45h17/StoreRate>

1. Overview

StoreRate is a role-based web application built with React + Vite, NestJS + TypeScript, PostgreSQL, and Prisma ORM.

The application supports three roles:

- **ADMIN** — manages users, stores, roles, and platform statistics.
- **USER** — searches stores and submits or updates ratings.
- **STORE_OWNER** — monitors assigned stores, ratings, and customer feedback.

Authentication and authorization use JWT with role-based route protection.

2. Key Features

Admin

- Secure administrator login and dashboard statistics.
- View total users, stores, ratings, and user-role statistics.
- Create users, administrators, store owners, and stores.
- Assign stores to store owners.
- Search and filter users; sort users ascending/descending.
- Search, filter, and sort stores; view user details and store-owner/rating information.
- Logout.

Normal User

- Registration and secure login.
- Store listing and search by name/address.
- View overall and personal ratings.
- Submit and modify 1–5 star ratings.
- Sort stores by name, address, and rating.
- Filter stores by All Stores, Rated by Me, and Not Rated.
- Rating descriptions such as Poor, Fair, Good, Very Good, and Excellent.
- Change password and logout.

Store Owner

- Secure owner login and dashboard.

- View owner information and assigned stores.
- View average rating and total ratings.
- View and sort customer ratings.
- View rating distribution from 1–5 stars.
- Change password and logout.

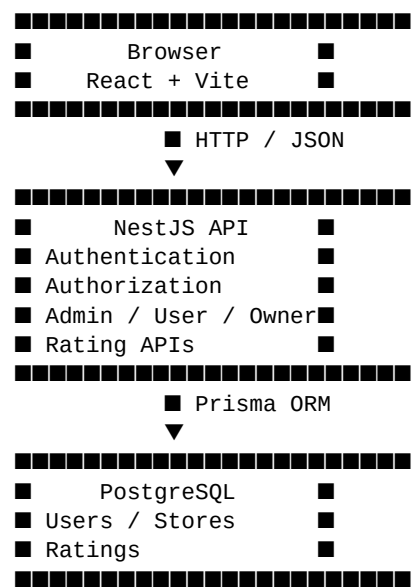
3. UI Improvements

- Modern StoreRate branding and logo across authentication and dashboard pages.
- Improved dashboard statistics.
- User rating filters and rating summary cards.
- Human-readable rating descriptions.
- Store-owner rating distribution.
- Consistent branding across login, registration, admin, user, and owner pages.
- Responsive and clearer organization of store and rating information.

4. Technology Stack

Area	Technologies
Frontend	React, Vite, React Router, Axios, Lucide React
Backend	NestJS, TypeScript, Prisma ORM, PostgreSQL
Authentication	JWT, Passport, bcrypt
Validation	class-validator
Tools	Node.js, npm, Git, GitHub, Docker, Docker Compose

5. Architecture



6. Authentication & Authorization

- Backend validates credentials.
- JWT access token is generated after successful login.
- Frontend stores authentication information.
- Axios sends the JWT using the Authorization header.
- NestJS guards validate tokens.
- Role-based protection restricts access to permitted dashboards and APIs.

```
ADMIN
USER
STORE_OWNER
```

7. Validation

- Name: minimum 20 characters, maximum 60 characters.
- Address: maximum 400 characters.
- Password: 8–16 characters, with at least one uppercase letter and one special character.
- Email: valid email format required.
- Rating: integer from 1 to 5.

8. Project Structure

```
StoreRate/
├── backend/
│   ├── prisma/
│   ├── src/
│   │   ├── admin/
│   │   ├── auth/
│   │   ├── owner/
│   │   ├── prisma/
│   │   └── users/
│   ├── package.json
│   └── README.md
├── frontend/
│   ├── src/
│   │   ├── assets/
│   │   │   └── storerate-logo.png
│   │   ├── components/
│   │   ├── context/
│   │   ├── pages/
│   │   │   ├── admin/
│   │   │   ├── owner/
│   │   │   └── user/
│   │   └── services/
│   ├── package.json
│   └── vite.config.js
├── ScreenShot/
├── docker-compose.yml
├── .gitignore
└── README.md
```

9. Local Setup

Prerequisites

- Node.js
- npm
- PostgreSQL
- Optional: Docker and Docker Compose

Backend

```
cd backend
npm install

# Create backend/.env
DATABASE_URL="postgresql://USERNAME:PASSWORD@localhost:5432/storerate?schema=public"
JWT_SECRET="your-development-secret"

npx prisma generate
npx prisma migrate dev
npm run start:dev
```

Backend default URL: **http://localhost:3000**

Frontend

```
cd frontend
npm install

# Create frontend/.env
VITE_API_URL=http://localhost:3000

npm run dev
```

The Vite terminal displays the frontend URL, normally **http://localhost:5173**.

10. Production Builds

```
# Backend
cd backend
npm run build

# Frontend
cd frontend
npm run build
```

11. Main API Routes

```
Authentication
POST /auth/login
POST /auth/register
POST /auth/change-password

User
GET /stores
POST /stores/:storeId/rating

Store Owner
GET /owner/dashboard

Admin
GET /admin/dashboard
GET /admin/users
GET /admin/users/:id
POST /admin/users
GET /admin/stores
POST /admin/stores
```

12. Database

StoreRate uses PostgreSQL with Prisma ORM.

User
Store
Rating

User
 ■■■ can submit Ratings
 ■■■ can own Stores

Store
 ■■■ has many Ratings

Rating
 ■■■ belongs to User
 ■■■ belongs to Store

A user's rating for a store can be updated rather than creating duplicate ratings for the same user/store combination.

13. Security

- JWT authentication.
- Role-based authorization.
- Protected frontend and backend routes.
- Password hashing with bcrypt.
- Input validation.
- Environment-based database configuration.
- Environment-based JWT secret.
- CORS configuration.

Sensitive configuration is excluded from Git using .gitignore. Actual .env files, dependencies, build output, and local generated/database data are not committed.

14. Demo Accounts

Role	Email	Password
Administrator	admin@storerate.local	Admin@1234
Store Owner	owner@storerate.local	Owner@1234
Normal User	Create through registration	Create through registration

Note: These credentials are intended for the local development/demo database only. Do not use them as production credentials.

15. Screenshots

The final implementation screenshots are stored in the repository's **ScreenShot/** directory and demonstrate the updated login, registration, admin, user, owner, and related application interfaces.

16. Testing Checklist

- Admin login
- User login
- Store Owner login
- User registration
- Admin creates users
- Admin creates store owners
- Admin creates stores
- Store assignment
- User store search
- User rating submission
- User rating modification
- Admin filtering
- Admin sorting
- User store sorting
- User rating filters
- Store Owner rating sorting
- Store Owner rating distribution
- Password change
- Logout
- Role-based route protection
- Updated branding
- Backend build
- Frontend build

17. GitHub Repository

Repository: <https://github.com/Ak45h17/StoreRate>

The repository contains the React frontend, NestJS backend, Prisma schema/migrations, PostgreSQL integration, Docker configuration, updated branding, final screenshots, and project documentation.

18. Coding Challenge

This project was completed as a full-stack coding challenge for Roxiler Systems. The implementation covers role-based authentication, user/store management, store ratings, rating modification, dashboards, search/filtering, sorting, validation, password management, and PostgreSQL persistence.

19. Submission Note

This document is supplementary project documentation. The primary submission is the GitHub repository containing the source code and final implementation.

20. License

This project was developed as a software engineering coding challenge and portfolio project.